

Week 2 Implementation Plan

OpenClaw + Stratus: Counterfactual Remediation Guardrail

Track

Stratus X1 Hackathon
Track: Multi-Agent Systems

0. Objective

Week 2 should deliver a **90% final product**, not another prototype.

By the end of this week, the demo should prove:

1. **Baseline vs Stratus**
 - the naive local reflex chooses the wrong action
 - Stratus rejects it by predicting a worse global consequence
2. **Real incident demo**
 - Scenario A is triggered live
 - the issue is observed through real dashboards and alerts
 - the safer action is executed and verified
3. **True multi-agent product**
 - the system is presented as named agents with distinct roles
 - OpenClaw is visibly coordinating them

Product claim:

We are building a Counterfactual Remediation Guardrail for operational agents: a system that predicts whether a remediation is safe before acting, then verifies whether the prediction held after action.

This should feel like a **world-model safety layer for operational agents**, not a generic incident copilot.

1. The Real Story We Are Demoing

Scenario A: Retail Death Spiral

It is a peak retail event, such as a flash sale.

Traffic surges. The payment service suffers a minor degradation, not a full outage. On its own, the event is survivable. But the checkout service is designed to be aggressively resilient: every timeout triggers retries. Those retries amplify load on the already-stressed payment dependency. Within minutes, the recovery mechanism itself becomes the failure engine.

This is what a real operator sees:

- checkout latency spikes past 2300ms
- error rate rises into the high teens
- retry volume surges far above normal baseline
- payment pods still appear “running,” but service quality is degraded
- dashboards show red latency and dependency stress across the checkout-payment path

This is what makes the scenario dangerous:

- the obvious human reflex is to restart payment pods
- that reflex is locally sensible
- but under retry amplification it can make recovery worse
- a traffic shift can also spread the problem into a second region

So the real operational question is:

| Which action is safe to take first, given the whole system state?

Action Strategy For Scenario A

We will not rank a fixed four-action list.

The product structure is:

1. **Action library**
 - a broader retail remediation catalog
 - target size: 12 to 20 actions
2. **Scenario shortlist**
 - Planner Agent selects the 4 to 6 actions that are actually relevant to the live incident
3. **Guardrail ranking**
 - Stratus ranks only the shortlist

This keeps the product realistic:

- richer than a toy demo
- focused enough for stable Stratus output
- clear enough for judges to follow

Retail Action Library Categories

- retry / overload control
- graceful degradation
- traffic management
- deployment rollback
- capacity / workload shaping

- last-resort service recovery

Default Scenario A Shortlist

- rate_limit_retries
- enable_payment_circuit_breaker
- increase_retry_backoff
- restart_payment
- shift_traffic
- disable_flag

Ground Truth

- restart_payment
 - tempting local reflex
 - globally dangerous under retry backlog
- shift_traffic
 - plausible
 - risks secondary-region spillover
- rate_limit_retries
 - breaks the feedback loop
 - is the safest first action
- enable_payment_circuit_breaker
 - valid containment option
 - safer than restart, but may be more user-visible than retry shaping
- increase_retry_backoff
 - valid containment option
 - improves pressure more slowly than rate limiting
- disable_flag
 - technically effective
 - too destructive for first response

What “Winning” Looks Like

The demo should clearly show:

- Baseline Agent chooses restart_payment
 - Guardrail Agent rejects it
 - Guardrail Agent chooses rate_limit_retries
 - OpenClaw executes the safer action
 - retry pressure and latency improve
 - the final report states what dangerous action was avoided
-

2. Real Issue Demo Path

Primary Demo Base

The primary live demo environment for Week 2 is:

- **OTel Demo on Kubernetes**

The point is to make the issue happen in a real system, not only in a synthetic local fixture.

How The Issue Is Triggered

Use the official OTel Demo feature-flag path to create the issue:

- open the OTel Demo feature UI at /feature
- turn on the payment-unreachable failure flag
 - paymentServiceUnreachable / paymentUnreachable depending on the OTel version
- turn on loadgeneratorFloodHomepage

This creates the closest live version of Scenario A:

- payment degrades
- checkout retries amplify the issue
- latency and retry alerts fire

How The Issue Is Observed

Use:

- Prometheus + Alertmanager
- OTel Demo dashboards
- browser-visible feature/UI state

The product should treat this as one incident state, not separate disconnected tools.

How The Issue Is Addressed

Execution path for Week 2:

- **Trigger path**
 - OTel Demo feature-flag UI
- **Observation path**
 - OTel Demo dashboards + Prometheus + Alertmanager
- **Remediation path**
 - OpenClaw browser executes the selected safe action through the remediation surface
- **Verification path**
 - Prometheus metrics + browser-visible state + final verdict report

Fallback

Keep the current local visual control plane only as a rehearsal / fallback path.

It should be framed in the plan as:

- fallback for reliability

- not the flagship Week 2 demo

Chaos Mesh

Chaos Mesh remains:

- optional stretch
 - not the primary Week 2 dependency
-

3. The Product As A Multi-Agent System

The product should be described and demoed as five named agents:

- **Sentinel Agent**
 - gathers alert, Prometheus, trace, and browser-visible evidence
- **Planner Agent**
 - selects the scenario shortlist from the retail action library
- **Challenger Agent**
 - computes the non-Stratus baseline choice
- **Guardrail Agent**
 - calls Stratus to simulate and rank futures before action
- **Operator + Verifier Agent**
 - uses OpenClaw browser/tools to execute the chosen action and confirm the outcome

Final Story Flow

1. **Sentinel Agent** observes the incident
2. **Planner Agent** selects the shortlist
3. **Challenger Agent** picks the obvious reflex
4. **Guardrail Agent** rejects it using Stratus
5. **Operator Agent** executes the safer move
6. **Verifier Agent** proves why that mattered

Required “Wow” Moment

The final demo must visibly show:

- the obvious action is `restart_payment`
 - the **Guardrail Agent** vetoes it
 - the product explains the predicted blast radius
 - the **Operator Agent** applies `rate_limit_retries`
 - the **Verifier Agent** shows retry pressure fell and blast radius stayed contained
-

4. Current Branches And Setup

Repo:

- <https://github.com/hanshengzhu0001/stratusxpennai>

Current / planned branches:

- workflow
- ranking+rollback
- baseline
- scenario+eval (planned)

Recommended Base Setup

```
git clone -b workflow
https://github.com/hanshengzhu0001/stratusxpennai.git
cd stratusxpennai
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
git fetch origin
```

Branch Usage

- workflow
 - shared integration
 - final verdict report
 - OpenClaw flow
- ranking+rollback
 - Stratus prompt/schema
 - veto logic
 - rollback reasoning
- baseline
 - baseline chooser
 - comparison artifact
- scenario+eval
 - scenario spec
 - evaluation rubric
 - visual acceptance criteria

Suggested Checkout

```
git checkout workflow
git checkout -b ranking+rollback origin/ranking+rollback
git checkout -b baseline origin/baseline
git checkout -b scenario+eval workflow
```

Shared Workflow Usage

```
.venv/bin/python run.py alerts/latest.json --phase plan
.venv/bin/python run.py alerts/latest.json --phase verify
```

OpenClaw Usage

- use this repo as the OpenClaw workspace
- run:
 - Use the `incident_guardrail` skill on the latest alert.

Independent Starting Points

- ranking work can start immediately on `ranking+rollback`
 - baseline work can start immediately on `baseline`
 - scenario/eval work can start immediately on `scenario+eval`
 - integration/report work continues on `workflow`
-

5. Current Status

We already have:

- a working end-to-end flow on `workflow`
- OpenClaw orchestration + browser handoff
- live Stratus ranking in the guarded decision step
- Prometheus-backed verification
- outputs for:
 - plan
 - browser playbook
 - final report

Week 1 contributions already established:

- workflow / integration backbone
- live Stratus path
- simulator/comparison shape
- Scenario A narrative and product framing

So Week 2 is not about inventing the project anymore.

Week 2 is about making one real incident story undeniable.

6. Team Rules For Week 2

- no overlap in **primary ownership**
- everyone touches both **OpenClaw** and **Stratus**
- everyone contributes in:
 - Develop
 - Test
 - Furnish

Primary ownership means one person is the final owner of that lane.

No one is assigned “generic workflow ownership.”

Instead, each person owns a distinct piece of the product and a distinct agent-facing responsibility.

7. Responsibilities

Hansen

Primary ownership:

- final product coherence
- final verdict report
- integration on workflow

Develop:

- integrate the five-agent story into the repo outputs and demo language
- merge action library, shortlist, baseline, Stratus, execution, and verification into one final verdict output
- make the final report judge-facing:
 - dangerous action rejected
 - why it was rejected
 - chosen safer action
 - predicted blast radius
 - actual blast radius avoided

Test:

- rehearse full end-to-end runs
- test OpenClaw browser reliability
- test final demo timing and fallback path

Furnish:

- final demo script
- final verdict report template
- final OpenClaw demo prompt

OpenClaw touchpoint:

- owns the Operator + Verifier product experience

Stratus touchpoint:

- owns how Guardrail output is rendered and explained in the product

Tony

Primary ownership:

- Guardrail Agent behavior
- Stratus schema, prompt, veto logic
- work on ranking+rollback

Develop:

- harden restart_payment vs rate_limit_retries
- make Stratus reliable on a 4 to 6 action shortlist, not a giant action dump
- make Stratus output stable on:
 - ranking
 - confidence
 - rationale
 - predicted latency delta
 - predicted retry reduction
 - predicted blast radius
 - rollback / veto reasoning

Test:

- repeated Stratus trials on the live Scenario A input
- compare prompt variants
- maintain 实验记录 TBD

Furnish:

- final request/response schema
- final prompt version
- one short “why restart is unsafe” explanation

OpenClaw touchpoint:

- ensure Guardrail output can directly drive OpenClaw browser execution and rollback messaging

Stratus touchpoint:

- deepest Stratus integration owner

Charlie**Primary ownership:**

- scenario truth
- evaluation rubric
- product positioning
- work on scenario+eval

Develop:

- write the canonical Scenario A spec
- define the retail action library categories and Scenario A shortlist policy
- define the action truth table
- define the exact visual cues the system must inspect before and after action
- define evaluation metrics:
 - decision fidelity
 - predicted-vs-actual drift
 - blast radius compression
 - dangerous reflex rejected score

Test:

- validate that the live OTel-triggered demo matches the intended story
- validate that the report tells the right business and operational narrative

Furnish:

- scenario spec
- evaluation rubric
- judge-facing product narrative
- “why this category is new” positioning copy

OpenClaw touchpoint:

- defines what Sentinel and Verifier must inspect in the UI

Stratus touchpoint:

- defines the truths and failure physics the Guardrail must reason over

Eason

Primary ownership:

- Challenger Agent
- baseline path
- comparison artifact
- work on baseline

Develop:

- implement `baseline.py`
- make baseline choose from the same Scenario A shortlist that Guardrail sees
- support:
 - restart-first baseline
 - plain LLM no-Stratus baseline
- generate comparison output:
 - baseline choice
 - Stratus choice
 - ground-truth best choice
 - which action is safer
 - blast radius difference
 - predicted-vs-actual gap

Important default:

- this work must be independent of integration work
- the baseline branch should be buildable and testable using the existing incident input shape and candidate action schema

Test:

- confirm baseline tends to choose `restart_payment` under Scenario A
- confirm Stratus path beats baseline on the same input
- confirm comparison artifact is understandable in one screen

Furnish:

- baseline module
- comparison artifact
- one short baseline-vs-Stratus explanation

OpenClaw touchpoint:

- make baseline results visible in the final OpenClaw-driven report and demo narrative

Stratus touchpoint:

- compare Challenger output directly against Guardrail output on the same scenario
-

8. Locked Interfaces

Use these as stable interfaces:

- incident state
- action library
- candidate actions
- planner shortlist
- baseline output
- Stratus ranking output
- final verdict report

Required final report fields:

- baseline_choice
 - guardrail_choice
 - ground_truth_best_action
 - dangerous_action_rejected
 - predicted_blast_radius
 - actual_blast_radius
 - blast_radius_avoided
 - decision_fidelity
 - predicted_vs_actual_drift
-

9. Test Plan And Demo Scenarios

Primary Live Demo Test

- deploy OTel Demo on K8s
- trigger the payment-unreachable flag in the OTel feature UI
- trigger loadgeneratorFloodHomepage
- confirm latency and retry alerts fire
- run:
 - Sentinel -> Planner -> Challenger -> Guardrail
- confirm baseline recommends restart_payment

- confirm Guardrail ranks `rate_limit_retries` first
- use OpenClaw browser to apply the safer action
- verify retry factor drops, alerts resolve or narrow, and the verdict report states the avoided catastrophe

Minimum Acceptance Criteria

- the dangerous action is explicitly shown and rejected
- Stratus reasoning is visibly necessary
- OpenClaw visibly acts in the UI
- Prometheus/browser verify the result
- the final output reads like a verdict, not debug output

Fallback Rehearsal

- keep the current local visual control plane as a fallback
 - frame it only as a rehearsal backup, not the flagship demo
-

10. Assumptions And Defaults

- primary Week 2 demo base: **OTel Demo on Kubernetes**
 - primary issue trigger: **OTel Demo feature flags**
 - primary real issue mechanics: **payment degradation + retry amplification**
 - primary differentiator: **Stratus vetoes the dangerous local reflex**
 - primary multi-agent story:
 - **Sentinel / Planner / Challenger / Guardrail / Operator+Verifier**
 - recommended new branch: `scenario+eval`
 - Chaos Mesh remains optional stretch work, not the core Week 2 dependency
-

11. Grounding / Sources

The PDF should explicitly cite the official sources that justify the “real issue” path:

- OpenTelemetry Demo repo / deployment baseline
 - <https://github.com/open-telemetry/opentelemetry-demo>
- OTel Demo feature-flag UI release note
 - <https://github.com/open-telemetry/opentelemetry-demo/releases>
- Prometheus + Alertmanager alerting model
 - <https://prometheus.io/docs/alerting/latest/alertmanager/>